



คู่มือสรุปพิมพ์ A4 พกเข้าห้องสอบฉบับสมบูรณ์ (Line-by-Line Code & Full AVR Assembly)

รวบรวมชุดคำสั่ง AVR Assembly ครบทุกคำสั่ง และอธิบายโค้ดตัวอย่างบรรทัดต่อบรรทัด (Line-by-Line Explanations) สำหรับเปิดดูทำข้อสอบได้อย่างแม่นยำ 100%



PART A: HARDWARE REFERENCE (หมวดฮาร์ดแวร์ — ฟิน รีจิสเตอร์ & SREG Flags)

1.1 สรุปฟิน ATmega328P, รีจิสเตอร์พอร์ต I/O (DDR_x, PORT_x, PIN_x) & ฟิน SREG

HARDWARE 1

พอร์ต	DDR _x (ทิศทาง)	PORT _x (เอาต์พุต)	PIN _x (อินพุต)	หน้าที่ลอจิกบิต
Port B, C, D	DDRB (0x24) DDRC (0x27) DDRD (0x2A)	PORTB (0x25) PORTC (0x28) PORTD (0x2B)	PINB (0x23) PINC (0x26) PIND (0x29)	- DDR_x Bit = 1: กำหนดพินเป็น Output - DDR_x Bit = 0: กำหนดพินเป็น Input - PORT_x Bit = 1 (Out): จ่ายไฟ HIGH (+5V) - PORT_x Bit = 1 (In): เปิด Internal Pull-up



ธง Status Register (SREG): [Bit 7: I (Interrupt) | Bit 6: T (Bit Copy) | Bit 5: H (Half Carry) | Bit 4: S (Sign) | Bit 3: V (Overflow: Cin(MSB) ⊕

Cout(MSB)) | Bit 2: N (Negative) | Bit 1: Z (Zero) | Bit 0: C (Carry)]



2.1 คลังชุดคำสั่งภาษา AVR Assembly ครบทุกคำสั่ง (All 40+ Instructions Reference Table)

ALL INSTRUCTIONS

หมวดคำสั่ง	คำสั่ง (Opcode)	รูปแบบ (Syntax)	การทำงาน (Operation) & รายละเอียดการอธิบาย	ธงที่ส่งผล (SREG)
1. ย้ายข้อมูล (Data Transfer)	LDI	LDI Rd, K	โหลดค่าคงที่ K (0-255) ลงรีจิสเตอร์ Rd (R16 ถึง R31) Direct Immediate	ไม่มี
	MOV / MOVW	MOV Rd, Rr	คัดลอกข้อมูลจาก Rr ไปยัง Rd ('MOVW' คัดลอกคู่รีจิสเตอร์ 16 บิต)	ไม่มี
	LDS / STS	LDS Rd, k/STS k, Rr	LDS อ่านข้อมูลจากแอดเดรส k ใน SRAM ลง Rd / STS เขียนค่าจาก Rr ลง SRAM	ไม่มี
	IN / OUT	IN Rd, A/OUT A, Rr	IN อ่านค่าจาก I/O Register A ลง Rd / OUT ส่งค่าจาก Rr ออก I/O Register A	ไม่มี
	PUSH / POP	PUSH Rr / POP Rd	PUSH เก็บข้อมูลลง Stack Pointer / POP ดึงข้อมูลออกจาก Stack คืนลง Rd	ไม่มี
	LPM	LPM Rd, Z	อ่านข้อมูล 1 ไบต์จาก Flash ROM ที่ชี้โดยพอยเตอร์ Z (R31:R30) ลง Rd	ไม่มี
	LD / ST	LD Rd, X/ST X, Rr	อ่าน/เขียนข้อมูล SRAM แบบอ้อมผ่าน Pointer X, Y, Z (รองรับ '+', '-' Pre/Post)	ไม่มี
	CLR / SER	CLR Rd/SER Rd	CLR เคลียร์ค่าใน Rd เป็น 0x00 (คือ 'EOR Rd, Rd') / SER เซตค่าเป็น 0xFF	Z=1 (CLR)
2. คำนวณ ALU (Arithmetic)	ADD / ADC	ADD Rd, Rr	บวก Rd = Rd + Rr ('ADC' บวกผลรวมตัวทด Carry Flag C เพิ่มอีก 1)	Z, C, N, V, H, S
	SUB / SUBI	SUB Rd, Rr	ลบ Rd = Rd - Rr ('SUBI' ลบด้วยค่าคงที่ K: Rd = Rd - K)	Z, C, N, V, H, S
	SBC / SBCI	SBC Rd, Rr	ลบพร้อมตัวยืม Carry: Rd = Rd - Rr - C ('SBCI' ลบค่าคงที่ K และ C)	Z, C, N, V, H, S
	INC / DEC	INC Rd/DEC Rd	เพิ่มค่า ('INC') หรือ ลดค่า ('DEC') ใน Rd ที่ละ 1 (ไม่กระทบ Carry Flag C)	Z, N, V, S
	MUL / MULS	MUL Rd, Rr	คูณเลข 8 บิต ผลลัพธ์ 16 บิตเก็บบนคู่ R1:R0 ('MULS' คัดเครื่องหมาย 2's)	Z, C
	NEG	NEG Rd	หาค่า 2's Complement ของ Rd (คือ Rd = 0 - Rd เปลี่ยนเครื่องหมายบวก/ลบ)	Z, C, N, V, H, S
	CP / CPC / CPI	CPI Rd, K	เปรียบเทียบ Rd กับ K (คือทดสอบลบ Rd - K เพื่อเซตธง SREG โดยไม่แก้ค่า Rd)	Z, C, N, V, H, S
3. ลอจิก & จัดการ บิต (Logic & Bit)	AND / ANDI	ANDI Rd, K	ลอจิก AND บิตต่อบิต ('ANDI' ใช้กับค่าคงที่ Mask บิตตัดส่วนไม่ต้องการออก)	Z, N, V=0, S
	OR / ORI	ORI Rd, K	ลอจิก OR บิตต่อบิต ('ORI' ใช้เซตบิตเฉพาะตำแหน่งให้เป็น 1)	Z, N, V=0, S
	EOR / COM	EOR Rd, Rr	'EOR' XOR บิตต่อบิต / 'COM' กลับบิตทุกบิต 1's Complement (Rd = 0xFF - Rd)	Z, N, V=0, S
	SBI / CBI	SBI A, b/CBI A, b	SBI เซตบิตที่ b ใน I/O A ให้เป็น 1 / CBI เคลียร์บิตที่ b ให้เป็น 0	ไม่มี
	SBIS / SBIC	SBIS A, b	SBIS ข้าม 1 คำสั่งถัดไปถ้าบิต b ใน I/O A เป็น 1 / SBIC ข้ามถ้าเป็น 0	ไม่มี
	SBRS / SBRC	SBRS Rr, b	ข้าม 1 คำสั่งถัดไปถ้าบิต b ในรีจิสเตอร์ Rr เป็น 1 ('SBRS') หรือเป็น 0 ('SBRC')	ไม่มี
4. กระโดด & ควบคุม (Branch & Jump)	JMP / RJMP	RJMP label	กระโดดไปยัง label ไร้เงื่อนไข ('RJMP' กระโดดสัมพันธ์ระยะทางใกล้ -2K ถึง +2K)	ไม่มี
	CALL / RET	CALL Subroutine	CALL เรียกฟังก์ชันย่อย (บันทึก PC ลง Stack) / RET ดึง PC คืนจาก Stack	ไม่มี
	BREQ / BRNE	BRNE label	BREQ กระโดดเมื่อ Z=1 (เท่ากัน) / BRNE กระโดดเมื่อ Z=0 (ไม่เท่ากัน)	ไม่มี
	BRLO / BRSH	BRLO label	BRLO กระโดดเมื่อ C=1 (น้อยกว่า) / BRSH กระโดดเมื่อ C=0 (มากกว่าหรือเท่ากับ)	ไม่มี
	BMI / BPL	BMI label	BMI กระโดดเมื่อ N=1 (ผลลัพธ์ติดลบ) / BPL กระโดดเมื่อ N=0 (ผลลัพธ์เป็นบวก)	ไม่มี
	BRCS / BRCC	BRCS label	BRCS กระโดดเมื่อ Carry C=1 / BRCC กระโดดเมื่อ Carry C=0	ไม่มี
	RETI	RETI	ดึง PC คืนจาก Stack หลังจบฟังก์ชันขัดจังหวะ (Interrupt Service Routine) + SEI	I=1
5. เลื่อน & หมุนบิต (Shift & Rotate)	LSL / LSR	LSL Rd/LSR Rd	LSL เลื่อนบิตไปทางซ้าย 1 บิต (คูณ 2) / LSR เลื่อนบิตไปทางขวา 1 บิต (หาร 2)	Z, C, N, V, S
	ROL / ROR	ROL Rd/ROR Rd	หมุนบิตผ่าน Carry Flag C (ROL ดึง C เข้าบิต 0, ROR ดึง C เข้าบิต 7)	Z, C, N, V, S
	ASR	ASR Rd	เลื่อนบิตไปทางขวาแบบคงเครื่องหมาย MSB ไว้ (Arithmetic Shift Right สำหรับเลขติดลบ)	Z, C, N, V, S
	SWAP	SWAP Rd	สลับบิตระหว่าง 4 บิตสูง (High Nibble) และ 4 บิตต่ำ (Low Nibble) ของ Rd	ไม่มี
6. ควบคุมระบบ (System Control)	SEI / CLI	SEI / CLI	SEI เปิดรับสัญญาณขัดจังหวะ Global Interrupt (I=1) / CLI ปิดการขัดจังหวะ (I=0)	I (Set/Clear)

หมวดคำสั่ง	คำสั่ง (Opcode)	รูปแบบ (Syntax)	การทำงาน (Operation) & รายละเอียดการอธิบาย	ช่องที่ส่งผล (SREG)
	NOP	NOP	No Operation (ไม่ทำอะไรเลย เสียเวลา 1 คาบ Clock Cycle ใช้หน่วยเวลา)	ไม่มี

2.1.1 ตารางถอดรหัสบิต Opcode และ Operand 16 บิตของ AVR (16-Bit Instruction Bit-Encoding)

OPCODE BIT MAP

จำแนกบิต Opcode (รหัสคำสั่ง), บิตระบุรีจิสเตอร์เป้าหมาย (d), รีจิสเตอร์ต้นทาง (r), แอดเดรส I/O (A), ค่าคงที่ (K), ตำแหน่งบิต (b), และ Relative Offset (k) ในคำสั่งขนาด 16 บิต (Bit 15 ถึง Bit 0):

หมวดคำสั่ง	คำสั่ง (Instruction)	โครงสร้างรหัสบิต 16 บิต (16-Bit Opcode Encoding)	การกระจายบิต Opcode (Bit 15 - Bit 0)	รายละเอียดความหมายของบิตแต่ละส่วน
คณิตศาสตร์ 2 รีจิสเตอร์	ADD Rd, Rr	0000 11rd dddd rrrr	[15:10] Opcode 000011 [9,3:0] Rr r rrrr [8:4] Rd d dddd	• Bit 15-10: Opcode คำสั่ง ADD (000011) • Bit 8-4: Rd 5 บิต (\$R0-R_31\$) • Bit 9,3-0: Rr 5 บิต (\$R0-R_31\$)
	SUB Rd, Rr	0001 10rd dddd rrrr	[15:10] Opcode 000110 [9,3:0] Rr r rrrr [8:4] Rd d dddd	• Bit 15-10: Opcode คำสั่ง SUB (000110) • Bit 8-4: Rd 5 บิต • Bit 9,3-0: Rr 5 บิต
โหลดค่าคงที่	LDI Rd, K	1110 KKKK dddd KKKK	[15:12] Opcode 1110 [11:8] K7-K4 KKKK [7:4] Rd dddd [3:0] K3-K0 KKKK	• Bit 15-12: Opcode คำสั่ง LDI (1110) • Bit 11-8: ค่าคงที่ K 4 บิตสูง (K7-K4) • Bit 7-4: Rd 4 บิต (R16 ถึง R31) • Bit 3-0: ค่าคงที่ K 4 บิตต่ำ (K3-K0)
อ่าน/เขียน I/O	IN Rd, A	1011 0AA dddd AAAA	[15:11] Opcode 10110 [10:9,3:0] Address A AAAAAA [8:4] Rd dddd	• Bit 15-11: Opcode คำสั่ง IN (10110) • Bit 10-9,3-0: I/O Address A 6 บิต (0x00-0x3F) • Bit 8-4: Rd 5 บิต (\$R0-R_31\$)
	OUT A, Rr	1011 1AAr rrrr AAAA	[15:11] Opcode 10111 [10:9,3:0] Address A AAAAAA [8:4] Rr rrrrr	• Bit 15-11: Opcode คำสั่ง OUT (10111) • Bit 10-9,3-0: I/O Address A 6 บิต (0x00-0x3F) • Bit 8-4: Rr 5 บิต (\$R0-R_31\$)
จัดการบิต I/O	SBI A, b	1001 1010 AAAA Abbb	[15:8] Opcode 10011010 [7:3] Address A AAAAA [2:0] Bit pos b bbb	• Bit 15-8: Opcode คำสั่ง SBI (10011010) • Bit 7-3: I/O Address A 5 บิต (0x00-0x1F) • Bit 2-0: ตำแหน่งบิต b 3 บิต (0 ถึง 7)
	CBI A, b	1001 1000 AAAA Abbb	[15:8] Opcode 10011000 [7:3] Address A AAAAA [2:0] Bit pos b bbb	• Bit 15-8: Opcode คำสั่ง CBI (10011000) • Bit 7-3: I/O Address A 5 บิต (0x00-0x1F) • Bit 2-0: ตำแหน่งบิต b 3 บิต (0 ถึง 7)
กระโดดเงื่อนไข	BREQ k	1111 00kk kkkk k000	[15:10] Opcode 111100 [9:3] Offset k kkkkkkk [2:0] Flag \$Z=1\$ 000	• Bit 15-10: Opcode คำสั่ง BRBS (111100) • Bit 9-3: Relative Offset k คัดเครื่องหมาย 7 บิต • Bit 2-0: Flag Bit (000 = ธง Z=1)
	BRNE k	1111 01kk kkkk k000	[15:10] Opcode 111101 [9:3] Offset k kkkkkkk [2:0] Flag \$Z=0\$ 000	• Bit 15-10: Opcode คำสั่ง BRBC (111101) • Bit 9-3: Relative Offset k คัดเครื่องหมาย 7 บิต • Bit 2-0: Flag Bit (000 = ธง Z=0)
กระโดด สัมพันธ์	RJMP k	1100 kkkk kkkk kkkk	[15:12] Opcode 1100 [11:0] Offset k kkkkkkkkkkkk	• Bit 15-12: Opcode คำสั่ง RJMP (1100) • Bit 11-0: Relative Offset k 12 บิต (-2048 ถึง +2047)
	RCALL k	1101 kkkk kkkk kkkk	[15:12] Opcode 1101 [11:0] Offset k kkkkkkkkkkkk	• Bit 15-12: Opcode คำสั่ง RCALL (1101) • Bit 11-0: Relative Offset k 12 บิต (-2048 ถึง +2047)



2.2 พิมพ์เขียวซอร์สโค้ดพร้อมคำอธิบายการทำงานบรรทัดต่อบรรทัด (Line-by-Line Code Explanations)

LINE BY LINE

1. การตั้งค่า Stack Pointer (SPH:SPL = RAMEND)

```
.INCLUDE "M328PDEF.INC"
.ORG 0x0000
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16
```

💡 อธิบายการทำงานบรรทัดต่อบรรทัด:

- บรรทัด 1 (`.INCLUDE "M328PDEF.INC"`): ดึงไฟล์นิยามซีเรียสเตอร์และแอดเดรสมาตรฐานของ ATmega328P เข้ามาใช้งาน
- บรรทัด 2 (`.ORG 0x0000`): กำหนดจุดเริ่มต้นของโปรแกรมให้อยู่ที่แอดเดรสแรก 0x0000 ของ Flash ROM (Reset Vector)
- บรรทัด 3 (`LDI R16, HIGH(RAMEND)`): ตั้งค่า 8 บิตสูงของแอดเดรสสูงสุด SRAM (RAMEND = 0x08FF → HIGH คือ 0x08) ลง R16
- บรรทัด 4 (`OUT SPH, R16`): ส่งค่า 0x08 ไปยัง Stack Pointer High Register (SPH)
- บรรทัด 5 (`LDI R16, LOW(RAMEND)`): ตั้งค่า 8 บิตต่ำของแอดเดรสสูงสุด SRAM (LOW คือ 0xFF) ลง R16
- บรรทัด 6 (`OUT SPL, R16`): ส่งค่า 0xFF ไปยัง Stack Pointer Low Register (SPL) เพื่อให้ Stack เริ่มต้นที่ปลายสุด 0x08FF

2. การวนลูปนับเลข 1 ถึง 10 ออกพอร์ต PORTB

```
LDI R16, 0xFF
OUT DDRB, R16
LDI R17, 1

COUNT_LOOP:
    OUT PORTB, R17
    CPI R17, 10
    BREQ LOOP_DONE
    INC R17
    RJMP COUNT_LOOP
LOOP_DONE:
```

💡 อธิบายการทำงานบรรทัดต่อบรรทัด:

- บรรทัด 1 (`LDI R16, 0xFF`): โหลดค่า 0xFF (11111112) ลง R16 เพื่อเตรียมตั้งค่าทุกพินเป็นเอาต์พุต
- บรรทัด 2 (`OUT DDRB, R16`): ส่งค่า 0xFF ไปยัง Data Direction Register Port B (DDRB) เพื่อตั้งค่าขา PB0-PB7 เป็น Output
- บรรทัด 3 (`LDI R17, 1`): กำหนดค่าเริ่มต้นของตัวนับใน R17 ให้เท่ากับ 1
- บรรทัด 4 (`COUNT_LOOP:`): ป้ายกำกับ (Label) จุดเริ่มต้นของลูปการนับ
- บรรทัด 5 (`OUT PORTB, R17`): ส่งค่านับปัจจุบันใน R17 ออกไปแสดงผลทางขาพอร์ต PORTB
- บรรทัด 6 (`CPI R17, 10`): เปรียบเทียบค่าใน R17 กับ 10 (ทดลองลบ R17 - 10 ถ้าเท่ากัน จะเซตธง Zero Flag Z=1)
- บรรทัด 7 (`BREQ LOOP_DONE`): ถ้า Z=1 (นับครบ 10 เท่ากันแล้ว) ให้กระโดดไปที่ป้าย LOOP_DONE เพื่อจบลูป
- บรรทัด 8 (`INC R17`): เพิ่มค่านับใน R17 ขึ้นทีละ 1 ($R17 = R17 + 1$)
- บรรทัด 9 (`RJMP COUNT_LOOP`): กระโดดกลับไปที่บรรทัดแรกของลูป COUNT_LOOP อีกครั้ง

3. การอ่านปุ่มกด Active Low สวิตช์ PINCO ควบคุมไฟ LED PORTB5

```

SBI DDRB, 5
CBI DDRC, 0
SBI PORTC, 0

READ_SWITCH:
    SBIC PINC, 0
    RJMP TURN_OFF
TURN_ON:
    SBI PORTB, 5
    RJMP READ_SWITCH
TURN_OFF:
    CBI PORTB, 5
    RJMP READ_SWITCH

```

💡 อธิบายการทำงานของบรรทัดต่อบรรทัด:

- **บรรทัด 1 (SBI DDRB, 5):** Set Bit 5 ของ DDRB ให้เป็น 1 เพื่อตั้งค่า PB5 เป็นเอาต์พุตสำหรับต่อไฟ LED
- **บรรทัด 2 (CBI DDRC, 0):** Clear Bit 0 ของ DDRC ให้เป็น 0 เพื่อตั้งค่า PC0 เป็นอินพุตสำหรับต่อปุ่มกด
- **บรรทัด 3 (SBI PORTC, 0):** Set Bit 0 ของ PORTC ให้เป็น 1 ขณะอยู่โหมด Input เพื่อเปิดใช้งาน Internal Pull-up Resistor
- **บรรทัด 5 (SBIC PINC, 0):** เช็คว่า PC0 ถ้าปุ่มกดถูกกดลงดิน (เป็น 0) ให้ข้ามคำสั่งบรรทัดถัดไป / ถ้ายังไม่ได้กด (เป็น 1) ทำบรรทัดถัดไป
- **บรรทัด 6 (RJMP TURN_OFF):** ทำงานเมื่อไม่ได้กดปุ่ม ให้กระโดดไปดับไฟ LED
- **บรรทัด 8 (SBI PORTB, 5):** ทำงานเมื่อกดปุ่ม (เพราะถูกข้ามมา) ให้ส่งไฟ 5V ออกขา PB5 ติดไฟ LED
- **บรรทัด 9 (RJMP READ_SWITCH):** กระโดดกลับไปอ่านค่าสวิตช์วนลูปต่อเนื่อง

📁 4. อัลกอริทึมแปลง Hex เป็น BCD 2 หลักด้วยวิธีลบซ้ำด้วย 10

```

LDI R16, 0x45
CLR R17

BCD_CONVERT:
    CPI R16, 10
    BRLO BCD_DONE
    SUBI R16, 10
    INC R17
    RJMP BCD_CONVERT

BCD_DONE:
    SWAP R17
    OR R17, R16
    OUT PORTB, R17

```

💡 อธิบายการทำงานของบรรทัดต่อบรรทัด:

- **บรรทัด 1 (LDI R16, 0x45):** โหลดค่าคงที่ Hex 0x45 (69 ทศนิยม) เข้า R16 เป็นข้อมูลอินพุตเริ่มต้น
- **บรรทัด 2 (CLR R17):** ล้างค่า R17 เป็น 0x00 สำหรับใช้นับจำนวนหลักสิบ (Tens digit)
- **บรรทัด 4 (CPI R16, 10):** เปรียบเทียบค่าที่เหลือใน R16 กับ 10
- **บรรทัด 5 (BRLO BCD_DONE):** ถ้าน้อยกว่า 10 (เกิด Carry C=1) ให้กระโดดออกจากลูปไป BCD_DONE (ค่าที่เหลือใน R16 คือหลักหน่วย)
- **บรรทัด 6 (SUBI R16, 10):** ลบค่าคงที่ 10 ออกจาก R16 ($R16 = R16 - 10$)
- **บรรทัด 7 (INC R17):** เพิ่มค่านับหลักสิบใน R17 ขึ้นทีละ 1 ($R17 = R17 + 1$)
- **บรรทัด 8 (RJMP BCD_CONVERT):** กระโดดกลับไปวนลบด้วย 10 ต่อไป
- **บรรทัด 11 (SWAP R17):** สลับบิตของ R17 (เช่น จาก 0000 0110 เป็น 0110 0000) เพื่อย้ายหลักสิบไปไว้ 4 บิตสูง (High Nibble)
- **บรรทัด 12 (OR R17, R16):** รวมหลักสิบใน 4 บิตสูง และหลักหน่วยจาก R16 ใน 4 บิตต่ำ ได้เป็น BCD สมบูรณ์ (0x69)
- **บรรทัด 13 (OUT PORTB, R17):** ส่งค่า BCD 0x69 ออกแสดงผลทางพอร์ต PORTB

📁 5. โค้ดแล็บจริง: AssemblyBlink1.asm (ไฟกะพริบพร้อมกัน 2 พอร์ต)

```

;=====
; Title: AssemblyBlink1.asm
; Created: 11/22/2013
; Author: khatathapswatdipisal
; Description: Blink all LEDs on PORTB and PORTD simultaneously
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF      ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16      ; Configure Port B pins as Outputs
    out DDRD, r16      ; Configure Port D pins as Outputs
    ldi r16, 0x00      ; Clear R16 to 0x00

loop:
    com r16             ; Complement R16 (toggle bits between 0x00 and 0xFF)
    out PORTB, r16      ; Write R16 to Port B
    out PORTD, r16      ; Write R16 to Port D
    call MY_DELAY       ; Delay for simulation timing
    rjmp loop           ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22             ; Decrement R22
    brne L3             ; Loop if R22 != 0
    dec r21             ; Decrement R21
    brne L2             ; Loop if R21 != 0
    dec r20             ; Decrement R20
    brne L1             ; Loop if R20 != 0
    ret                 ; Return from subroutine

```

 อธิบายการทำงานของบรรทัดต่อบรรทัด:

- **ldi r16, 0xFF & out DDRB/DDR, r16:** โหลด 0xFF ตั้งค่า PORTB และ PORTD ทั้งหมดเป็น Output
- **com r16:** กลับบิตทุกบิตใน R16 (สลับระหว่าง 0x00 → 0xFF → 0x00) ทำให้ไฟกะพริบติดดับสลับกันตามคาบเวลา
- **out PORTB/PORTD, r16:** ส่งค่า R16 ออกควบคุมไฟ LED บนทั้งสองพอร์ตพร้อมกัน
- **call MY_DELAY:** เรียกฟังก์ชันย่อยสร้างความหน่วงเวลาโดยใช้ลูปซ้อนกัน 3 ชั้น (R20, R21, R22) เพื่อให้ตามองเห็นการกะพริบ

 6. โค้ดแล็บจริง: AssemblyBlink1_modified.asm (ไฟกะพริบสลับพอร์ต PORTB ติดเมื่อ PORTD ดับ)

```

;=====
; Title: AssemblyBlink1_modified.asm
; Created: 11/22/2013 (Modified: 2026-07-19)
; Author: khatathapswatdipisal
; Description: Alternate blinking: Port B ON when Port D is OFF
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF      ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16      ; Configure Port B pins as Outputs
    out DDRD, r16      ; Configure Port D pins as Outputs
    ldi r16, 0x00      ; Clear R16 to 0x00

loop:
    com r16            ; Complement R16 (toggle R16 between 0x00 and 0xFF)
    out PORTB, r16     ; Write R16 directly to Port B (PB0-PB7)

    mov r17, r16       ; Copy R16 to R17
    com r17            ; Complement R17 (so R17 is always the opposite of R16)
    out PORTD, r17     ; Write R17 to Port D (PD0-PD7)

    call MY_DELAY      ; Delay for simulation timing
    rjmp loop          ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22            ; Decrement R22
    brne L3            ; Loop if R22 != 0
    dec r21            ; Decrement R21
    brne L2            ; Loop if R21 != 0
    dec r20            ; Decrement R20
    brne L1            ; Loop if R20 != 0
    ret               ; Return from subroutine

```

 อธิบายการทำงานของบรรทัดต่อบรรทัด:

• **com r16 & out PORTB, r16:** กลับบิต R16 และส่งออก PORTB (เช่น PB=0xFF ไฟติด)

• **mov r17, r16 & com r17 & out PORTD, r17:** ก๊อปปี้ R16 ลง R17 แล้วกลับบิต R17 เพื่อให้ R17 มีค่าตรงข้ามกับ R16 เสมอ (เช่น PD=0x00 ไฟดับ) ทำให้ทั้งสองพอร์ตรัดติดับสลับกัน